Real-world (rebuild Project 2 as a graph) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The exact pieces:

- **State:** `messages: Annotated[list, operator.add]` and `steps: Annotated[list, operator.add]` — each node returns *new* items, and LangGraph adds them to the old list (the reducer from this document's Basic exercise).
- **`think` node:** `ChatOpenAI(...).bind_tools(AGENT_TOOLS).invoke(state["messages"])` — return `{"messages": [response]}`.
- **`act` node:** loop over `state["messages"][-1].tool_calls`, call Doc07's `run_tool(call, AGENT_TOOLS)`, and return the `{"role": "tool", ...}` messages plus one step dict per call.
- **`route`:** `"act"` if the last message has `tool_calls`, otherwise `"finish"`, mapped to `END`.
- **Step limit:** `graph.invoke(start, config={"recursion_limit": 20})` — the graph's `max_iterations`.

Then ask the real question: does the graph behave the same as the loop? Answers are worded differently every run, so compare **which tools** each used, on the same prompts — and, if you want a second signal, how many model turns each took.

Hint 2 — The plan, and almost the whole thing

```
build_graph() -> compiled graph          (the Basic wiring, in a function)
run_graph(graph, task) -> final state    (recursion_limit=20)

compare_with_loop(graph):
    for each test prompt:
        loop_tools  = tool names from run_agent(prompt, 5).steps   (Doc07)
        graph_tools = tool names from run_graph(graph, prompt)["steps"]
        print MATCH or MISMATCH
```

```
    raise AssertionError if any prompt mismatched

count_model_turns(messages):
    count the messages that are NOT dicts (the model's replies)
```

One piece to get you unstuck:

```
def tool_names(steps: list[dict]) -> list[str]:
    names = []
    for step in steps:
        names.append(step["tool"])
    return names
```

Pick prompts with a known right tool: one calculation, one weather question, one general-knowledge question (no tool). Then compare against the Solution.

Full solution: Show me the solution